TF–IDF Vectorizer

From Word Counts to Document Similarity and Text Classification

Dr. Adnan Arefeen

North South University

Example

Consider three short documents:

Documents

D_1 : `cat likes milk`

D_2 : `cat likes fish`

D_3 : `dog likes fish`

The vocabulary is:

$$\{\texttt{cat}, \texttt{likes}, \texttt{milk}, \texttt{fish}, \texttt{dog}\}$$

Question

Which words are most useful for distinguishing the documents?

The Main Idea

Intuition

A word is important when it appears frequently in one document but does not appear frequently in every document.

$$\boxed{\text{TF–IDF} = \text{Local Importance} \times \text{Global Rarity}}$$

- **TF**: How important is the word inside this document?
- **IDF**: How unusual is the word across the whole document collection?

Before TF–IDF: Count Vectorizer

A Count Vectorizer represents each document using raw word counts.

Document	cat	likes	milk	fish	dog
D_1	1	1	1	0	0
D_2	1	1	0	1	0
D_3	0	1	0	1	1

Problem

The word `likes` appears in every document, but Count Vectorizer still gives it the same raw weight as more informative words such as `milk` or `dog`.

Step 1: Term Frequency (TF)

Term Frequency measures how often a term occurs in a particular document.

$$\boxed{TF(t, d) = \frac{\text{Number of occurrences of term } t \text{ in } d}{\text{Total number of terms in } d}}$$

For

$$D_1 = \texttt{cat likes milk},$$

there are 3 terms, so

$$TF(\texttt{cat}, D_1) = TF(\texttt{likes}, D_1) = TF(\texttt{milk}, D_1) = \frac{1}{3}.$$

Terms not present in the document have TF equal to 0.

TF Values for D_1

For `cat likes milk`:

Term	TF in D_1
cat	$1/3 \approx 0.333$
likes	$1/3 \approx 0.333$
milk	$1/3 \approx 0.333$
fish	0
dog	0

Observation

TF only considers one document. It does not know whether a word is common or rare across the entire corpus.

Step 2: Inverse Document Frequency (IDF)

IDF reduces the importance of terms that occur in many documents.

$$IDF(t) = \log\left(\frac{N}{df(t)}\right)$$

where:

- N = total number of documents;
- $df(t)$ = number of documents containing term t.

Interpretation

- Common across many documents $\Rightarrow$ small IDF.
- Rare across documents $\Rightarrow$ large IDF.

Example: IDF of a Very Common Word

The term `likes` appears in all three documents.

$$N = 3, \qquad df(\texttt{likes}) = 3$$

Therefore,

$$IDF(\texttt{likes}) = \log\left(\frac{3}{3}\right) = \log(1) = 0.$$

Meaning

Because `likes` appears everywhere, it has almost no power to distinguish one document from another under this simple IDF definition.

Example: IDF of a Rare Word

The term `milk` appears only in D_1.

$$N = 3, \qquad df(\texttt{milk}) = 1$$

Therefore,

$$IDF(\texttt{milk}) = \log\left(\frac{3}{1}\right) = \log(3) \approx 1.099.$$

Key Comparison

$$IDF(\texttt{milk}) > IDF(\texttt{likes}).$$

Thus, `milk` is more informative for distinguishing documents.

IDF for the Entire Vocabulary

For the three-document corpus:

Term	$df(t)$	$IDF(t) = \log(3/df(t))$
cat	2	0.405
likes	3	0
milk	1	1.099
fish	2	0.405
dog	1	1.099

Pattern

Rare terms such as `milk` and `dog` receive larger IDF values than common terms.

Step 3: Combine TF and IDF

TF–IDF is simply the product of TF and IDF:

$$\boxed{TFIDF(t, d) = TF(t, d) \times IDF(t)}$$

For $D_1 = \texttt{cat likes milk}$:

$$\begin{aligned} TFIDF(\texttt{cat}, D_1) &= \frac{1}{3}(0.405) \approx 0.135, \\ TFIDF(\texttt{likes}, D_1) &= \frac{1}{3}(0) = 0, \\ TFIDF(\texttt{milk}, D_1) &= \frac{1}{3}(1.099) \approx 0.366. \end{aligned}$$

TF–IDF Vector for D_1

Using the vocabulary order

$$[\texttt{cat}, \texttt{likes}, \texttt{milk}, \texttt{fish}, \texttt{dog}],$$

we obtain approximately:

$$\boxed{D_1 = [0.135,\ 0,\ 0.366,\ 0,\ 0]}$$

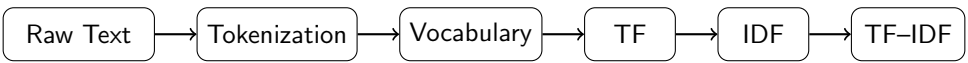
$$TFIDF(\texttt{milk}) > TFIDF(\texttt{cat}) > TFIDF(\texttt{likes})$$

Interpretation

`milk` is relatively rare and therefore receives the largest weight in D_1.

Why Is It Called a Vectorizer?

A vectorizer converts raw text into a numerical vector that machine-learning algorithms can process.



Example:

$$\texttt{``I love machine learning''} \quad \longrightarrow \quad [0,\ 0.72,\ 0,\ 0.51,\ 0.33, \ldots]$$

The exact dimensions correspond to vocabulary terms.

A More Realistic Example

Consider:

$$D_1 : \texttt{machine learning is useful}$$
$$D_2 : \texttt{machine learning is powerful}$$
$$D_3 : \texttt{deep learning is powerful}$$

- `is` appears in every document $\Rightarrow$ very low importance.
- `learning` appears in every document $\Rightarrow$ low discriminative power.
- `machine` appears in two documents $\Rightarrow$ moderate importance.
- `useful` appears only once $\Rightarrow$ high IDF.

Core Principle

TF–IDF emphasizes terms that are frequent locally but uncommon globally.

TF–IDF in scikit-learn

In practice, we normally use a library implementation.

```
from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "cat likes milk",
    "cat likes fish",
    "dog likes fish"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Why scikit-learn Values Differ from Manual Values

The default implementation uses a smoothed IDF formula:

$$\boxed{IDF(t) = \log\left(\frac{1+N}{1+df(t)}\right) + 1}$$

For `milk`:

$$IDF(\texttt{milk}) = \log\left(\frac{4}{2}\right) + 1 \approx 1.693.$$

For `likes`:

$$IDF(\texttt{likes}) = \log\left(\frac{4}{4}\right) + 1 = 1.$$

Also Important

scikit-learn normally applies vector normalization, so the final numeric values differ from simple hand calculations.

CountVectorizer vs. TfidfVectorizer

CountVectorizer

For

```
cat cat milk
```

a count vector may contain

$$[2, 1, 0, \ldots].$$

It only records raw frequency.

TfidfVectorizer

It considers both:

- frequency inside the current document;
- rarity across all documents.

TF–IDF = frequency here $\times$ rarity everywhere

Why TF–IDF Is Useful for Classification

Suppose we want to classify news articles into:

$$\text{Sports}, \quad \text{Politics}, \quad \text{Technology}.$$

Words such as

$$\texttt{the}, \texttt{is}, \texttt{and}, \texttt{this}$$

are poor features because they appear in many documents.
But terms such as

$$\texttt{goal}, \texttt{football}, \texttt{election}, \texttt{parliament}, \texttt{GPU}, \texttt{software}$$

can be highly discriminative.

Typical Pipeline

Text $\rightarrow$ TF–IDF $\rightarrow$ Logistic Regression / SVM / Naive Bayes $\rightarrow$ Class Prediction

TF–IDF and Cosine Similarity

After converting documents to vectors, we can compare their directions using cosine similarity:

$$\boxed{\text{cosine}(A, B) = \frac{A \cdot B}{\|A\|\|B\|}}$$

Example:

D_1 : `machine learning artificial intelligence`

D_2 : `machine learning algorithms`

D_3 : `football world cup`

We expect:

$$\textit{Similarity}(D_1, D_2) > \textit{Similarity}(D_1, D_3).$$

Cosine Similarity in Python

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

documents = [
    "machine learning artificial intelligence",
    "machine learning algorithms",
    "football world cup"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(documents)

similarity = cosine_similarity(X)
print(similarity)
```

A typical pattern is:

$$\textit{Similarity}(D_1, D_2) \text{ moderate/high}, \qquad \textit{Similarity}(D_1, D_3) \approx 0.$$

TF–IDF for Search and Information Retrieval

Suppose the document collection contains:

$$D_1 : \texttt{Python machine learning tutorial}$$
$$D_2 : \texttt{Java programming tutorial}$$
$$D_3 : \texttt{Deep learning with Python}$$

The user searches for:

$$Q = \texttt{Python learning}.$$

Procedure:

1. Transform documents and query into TF–IDF vectors.
2. Compute cosine similarity between the query and every document.
3. Rank documents by similarity score.

Likely ranking:

$$D_1 > D_3 > D_2.$$

Strengths of TF–IDF

- Simple to understand and implement.
- Fast and computationally inexpensive.
- Produces interpretable features.
- Strong baseline for many classification and retrieval tasks.
- Works well with linear models such as Logistic Regression and SVM.
- Does not require large neural networks or GPUs.

Main Limitation: No Semantic Understanding

Consider:

$$D_1 : \texttt{The automobile is fast.}$$
$$D_2 : \texttt{The car is quick.}$$

Humans recognize strong semantic similarity, but TF–IDF sees different tokens:

$$\texttt{automobile} \neq \texttt{car}, \qquad \texttt{fast} \neq \texttt{quick}.$$

Therefore, the computed similarity can be much lower than expected.

Reason

TF–IDF models lexical overlap, not true contextual meaning.

From TF–IDF to Modern Embeddings

A simplified historical progression is:

Important

TF–IDF remains valuable as a fast, interpretable, and competitive baseline even in modern NLP workflows.

Final Summary

The One Formula to Remember

$$\boxed{TFIDF(t,d) = TF(t,d) \times IDF(t)}$$

- TF measures importance inside one document.
- IDF penalizes words that occur in many documents.
- TF–IDF converts text into sparse numerical vectors.
- Cosine similarity can compare those vectors.
- TF–IDF is excellent for interpretable baselines, retrieval, and classical text classification.
- It does not directly understand synonyms, context, or semantic meaning.